

Loan Approval Prediction

Internship Task Submission | Zomato | Abhinav Gomra

Dataset	Kaggle — Loan Approval Prediction Case Study (614 rows, 12 features)
Task	Binary classification (Loan_Status: Y / N)
Models	Logistic Regression, Random Forest, Decision Tree
Imbalance	Class weights · SMOTE · Random undersampling
Notebook	loan_approval_prediction.ipynb
GitHub	https://github.com/abhinavgomra/Loan_approval_prediction

1. Objective

Build a supervised binary classifier to predict whether a loan application will be **approved (Y)** or **rejected (N)** based on borrower demographics, income, credit history, and loan details. The project covers the full ML pipeline: EDA → preprocessing → imbalance handling → model comparison → threshold optimisation → business recommendation.

2. Setup & Imports

■ Cell 1 — Imports &configuration

```
import warnings, matplotlib.pyplot as plt, numpy as np
import pandas as pd, seaborn as sns
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (ConfusionMatrixDisplay, RocCurveDisplay,
                             classification_report, f1_score, precision_score, recall_score, roc_auc_score)
from sklearn.model_selection import StratifiedKFold, cross_validate, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", palette="muted")
RANDOM_STATE = 42
DATA_PATH = Path("data/loan_prediction.csv")
FIG_DIR = Path("outputs/figures")
FIG_DIR.mkdir(parents=True, exist_ok=True)
```

3. Load & Explore Data

■ Cell 4 — Load CSV

```
df = pd.read_csv(DATA_PATH)
```

```
df.head()
```

First 5 rows of the raw dataset:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
0	LP001002	Male	No	0	Graduate	No	5849		0.0				
		NaN		360.0	1.0	Urban	Y						
1	LP001003	Male	Yes	1	Graduate	No	4583		1508.0				
		128.0		360.0	1.0	Rural	N						
2	LP001005	Male	Yes	0	Graduate	Yes	3000		0.0				
		66.0		360.0	1.0	Urban	Y						
3	LP001006	Male	Yes	0	Not Graduate	No	2583		2358.0				
		120.0		360.0	1.0	Urban	Y						
4	LP001008	Male	No	0	Graduate	No	6000		0.0				
		141.0		360.0	1.0	Urban	Y						

■ Cell 5 — Shape &summary

```
print("Shape:", df.shape)
print("\nMissing values:\n", df.isnull().sum().sort_values(ascending=False))
print("\nTarget distribution:\n",
df["Loan_Status"].value_counts())df["Loan_Status"].value_counts(normalize=True).mul(100
```

Shape: (614, 13)

Missing values:

Credit_History	50
Self_Employed	32
LoanAmount	22
Dependents	15
Loan_Amount_Term	14
Gender	13
Married	3

Target distribution:

Y	422
N	192

Y	68.7%
N	31.3%

4. Exploratory Data Analysis (EDA)

4.1 Target Class Balance

■ Cell 7 — Target balancechart

```
counts=df["Loan_Status"].value_counts() fig,
ax = plt.subplots(figsize=(5, 4))
sns.barplot(x=counts.index,y=counts.values,hue=counts.index,
            legend=False, ax=ax, palette="Set2")
ax.set_title("Loanapprovalvsrejection") for
i, v in enumerate(counts.values):
    ax.text(i,v+5,str(v),ha="center")plt.tight_layout(
)
plt.savefig(FIG_DIR / "01_target_balance.png",dpi=120)
plt.show()
```

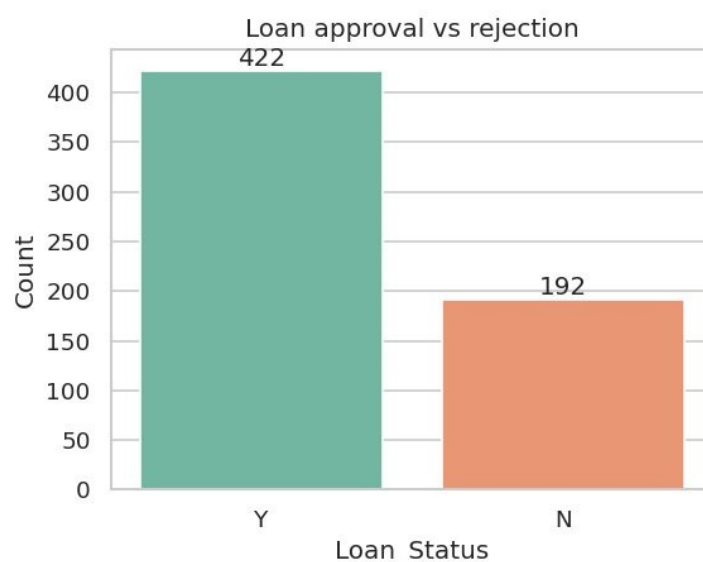


Fig 1 — Class imbalance: 422 approved (68.7%) vs 192 rejected (31.3%)

4.2 Missing Values

■ Cell 8 — Missing valueschart

```
missing=df.isnull().sum().sort_values(ascending=False)
missing = missing[missing > 0]
fig, ax = plt.subplots(figsize=(8,4))
sns.barplot(x=missing.index,y=missing.values,ax=ax,color="steelblue")
ax.set_title("Missing values per column")
ax.set_ylabel("Count")plt.xticks(rotation=45
,ha="right")plt.tight_layout()
plt.savefig(FIG_DIR / "02_missing_values.png",dpi=120)
```

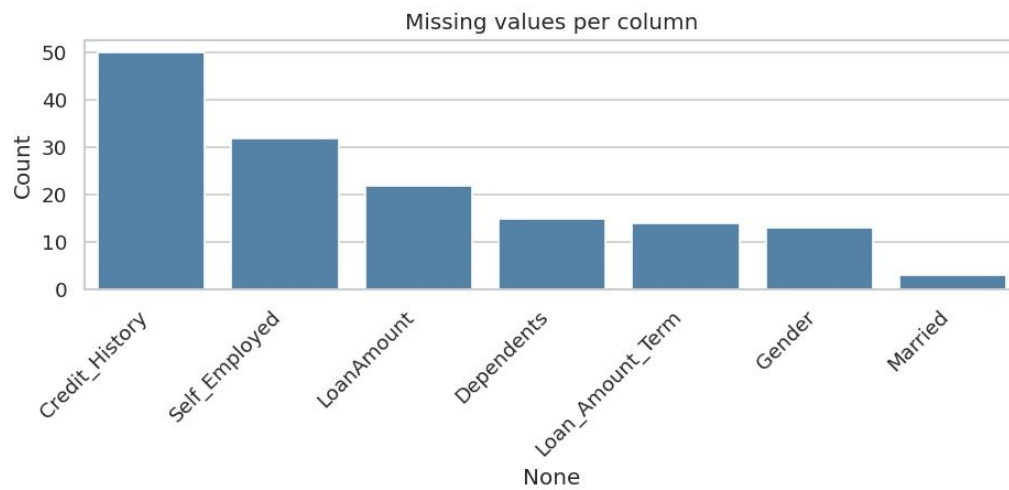


Fig 2 — Credit_History has the most missing values (50 rows); all handled via median/mode imputation

4.3 Total Income by Loan Outcome

Cell 9 — Incomeboxplot

```
plot_df = df.copy()
plot_df["TotalIncome"] = plot_df["ApplicantIncome"] + plot_df["CoapplicantIncome"].fillna(0)
fig, ax = plt.subplots(figsize=(7, 4))
sns.boxplot(data=plot_df, x="Loan_Status", y="TotalIncome",
            hue="Loan_Status", legend=False, ax=ax)
ax.set_title("Totalhouseholdincomebyloanoutcome")
plt.tight_layout()
plt.savefig(FIG_DIR / "03_income_by_status.png", dpi=120)
```

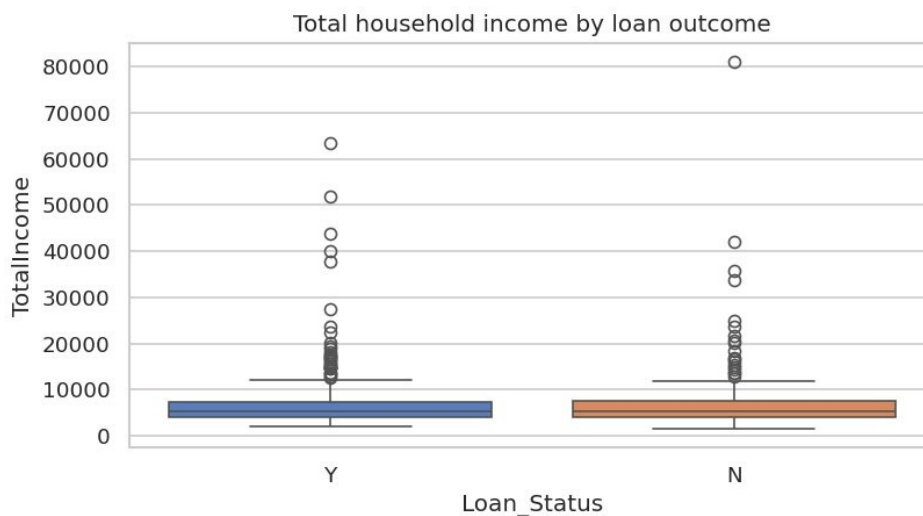


Fig 3 — Both groups share similar median incomes; approved group shows more high-income outliers

4.4 Credit History vs Approval Rate

Cell 10 — Credit history barchart

```
credit = (df.dropna(subset=["Credit_History"])
         .groupby("Credit_History")["Loan_Status"]
         .apply(lambda s: (s == "Y").mean())
         .reset_index(name="approval_rate"))
fig, ax = plt.subplots(figsize=(5, 4))
sns.barplot(data=credit, x="Credit_History", y="approval_rate", ax=ax, color="teal")
ax.set_ylim(0, 1)
ax.set_ylabel("Approval rate (Y)")
ax.set_title("Approvalratebycredithistory")
plt.tight_layout()
plt.savefig(FIG_DIR / "04_credit_history_approval.png", dpi=120)
```

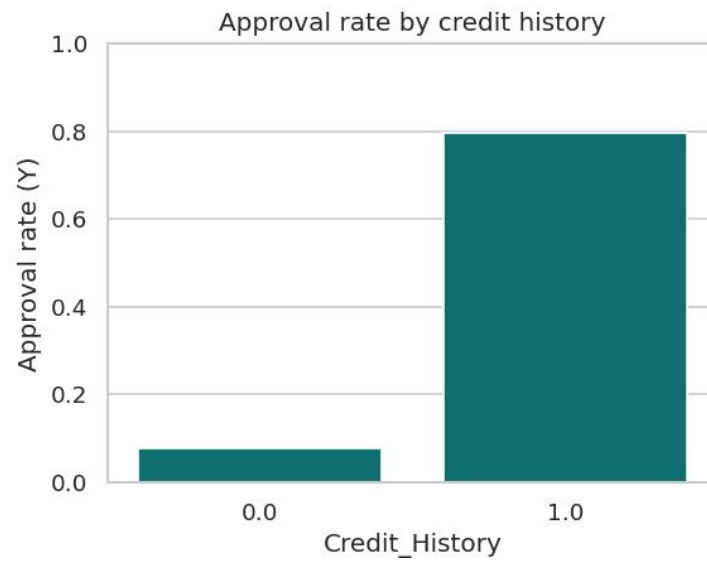


Fig 4 — Applicants with good credit history (1.0) have ~80% approval rate vs <10% without

4.5 Loan Amount by Property Area

Cell 11 — Loan amount by area

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.boxplot(data=df, x="Property_Area", y="LoanAmount",
            hue="Property_Area", legend=False, ax=ax)
ax.set_title("Requested loan amount by property area")
plt.tight_layout()
plt.savefig(FIG_DIR / "05_loan_amount_by_area.png", dpi=120)
```

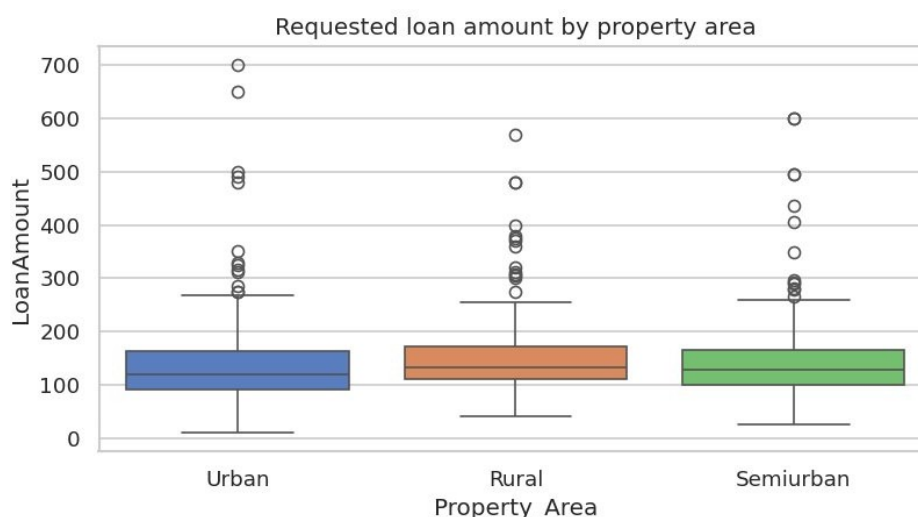


Fig 5 — Median loan amount is similar across areas; Rural has more high-value outliers

4.6 Correlation Matrix

Cell 12 — Correlation heatmap

```
num_cols=["ApplicantIncome", "CoapplicantIncome", "LoanAmount", "Loan_Amount_Term", "Credit_History"]
corr_df = df[num_cols].copy()
for c in num_cols:
    corr_df[c] = corr_df[c].fillna(corr_df[c].median())
corr_df["Approved"] = (df["Loan_Status"]=="Y").astype(int)
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(corr_df.corr(), annot=True, fmt=".2f", cmap="coolwarm", ax=ax)
ax.set_title("Correlation matrix (numeric features)")
plt.tight_layout()
plt.savefig(FIG_DIR / "06_correlation_heatmap.png", dpi=120)
```

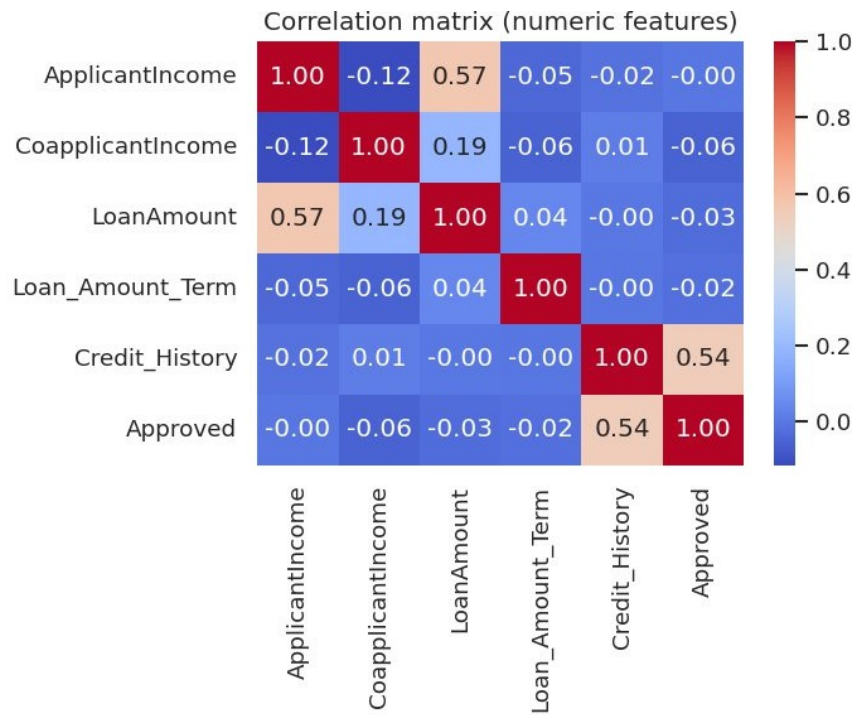


Fig 6 — Credit_History has the strongest correlation with Approved (0.54); income & loan amount correlated (0.57)

5. Preprocessing Pipeline

■ Cell 15 — Feature engineering & train/testsplit

```
TARGET,ID_COL="Loan_Status","Loan_ID" raw
= df.drop(columns=[ID_COL]).copy()
raw[TARGET] = (raw[TARGET] == "Y").astype(int)    # 1 =approved

cat_cols =["Gender","Married","Dependents","Education","Self_Employed","Property_Area"]
num_cols =["ApplicantIncome","CoapplicantIncome","LoanAmount","Loan_Amount_Term","Credit_History"]

#Imputation
processed=raw.copy() for
col in num_cols:
    processed[col]=processed[col].fillna(processed[col].median())
for col in cat_cols:
    processed[col] =processed[col].fillna(processed[col].mode(dropna=True).iloc[0])

X=processed.drop(columns=[TARGET]) y
= processed[TARGET]

X_train, X_test, y_train, y_test =train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE,stratify=y)
```

■ Cell 16 — ColumnTransformer (encode +scale)

```
preprocessor =
    ColumnTransformer([ ("num",StandardS
    caler(),num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore", drop="first"),cat_cols),
    ])
```

■ Cell 18 — Imbalance handler + modelbuilder

```
defmake_preprocessed_xy(X_part,y_part,strategy): pipe
    = Pipeline([("prep", preprocessor)])
X_mat=pipe.fit_transform(X_part,y_part) if
strategy == "smote":
    X_res,y_res=SMOTE(random_state=RANDOM_STATE).fit_resample(X_mat,y_part) elif
strategy == "undersample":
    X_res,y_res=RandomUnderSampler(random_state=RANDOM_STATE).fit_resample(X_mat,y_part)else:
    X_res,y_res=X_mat,y_part return
X_res, y_res, pipe

defbuild_models(class_weight=None):
    cw = class_weight
    return{
        "Logistic Regression": LogisticRegression(max_iter=2000,class_weight=cw,
                                                    random_state=RANDOM_STATE),
        "Decision Tree":      DecisionTreeClassifier(class_weight=cw,
                                                    random_state=RANDOM_STATE),
        "RandomForest":      RandomForestClassifier(n_estimators=200,class_weight=cw,
```

```
random_state=RANDOM_STATE),  
}
```

6. Model Training & Evaluation

■ Cell 20 — Train all 9 model-strategy combinations

```
def evaluate_model(name, model, X_tr, y_tr, X_te, y_te, strategy):
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    y_prob = model.predict_proba(X_te)[:, 1]
    return {
        "model": name, "imbalance_strategy": strategy,
        "precision": precision_score(y_te, y_pred, zero_division=0),
        "recall": recall_score(y_te, y_pred, zero_division=0),
        "f1": f1_score(y_te, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_te, y_prob),
    }

results, fitted = [], {}
for strategy in IMBALANCE_STRATEGIES:
    cw = "balanced" if strategy == "class_weight" else None
    X_tr, y_tr, prep_pipe = make_preprocessed_xy(X_train, y_train, strategy)
    X_te = prep_pipe.transform(X_test)
    for name, model in build_models(class_weight=cw).items():
        row = evaluate_model(name, model, X_tr, y_tr, X_te, y_test, strategy)
        results.append(row)
        fitted[(strategy, name)] = (model, prep_pipe)

metrics_df = pd.DataFrame(results).sort_values("f1", ascending=False)
```

6.1 Results Table (sorted by F1)

All 9 combinations evaluated on the held-out 20% test set. Best row highlighted.

Model	Strategy	Precision	Recall	F1	ROC-AUC
Logistic Regression	class_weight	0.875	0.858	0.867	0.857
Logistic Regression	smote	0.873	0.840	0.856	0.861
Logistic Regression	undersample	0.869	0.877	0.873	0.846
Random Forest	class_weight	0.831	0.925	0.875	0.814
Random Forest	smote	0.833	0.896	0.864	0.807
Random Forest	undersample	0.835	0.811	0.823	0.782
Decision Tree	smote	0.805	0.972	0.880	0.751
Decision Tree	undersample	0.806	0.783	0.794	0.711
Decision Tree	class_weight	0.837	0.679	0.750	0.762

6.2 Cross-Validation (5-fold) — Logistic Regression + class_weight

■ Cell 22 — Cross-validation

```
cv=StratifiedKFold(n_splits=5,shuffle=True,random_state=RANDOM_STATE) pipe_lr
= Pipeline([
    ("prep",preprocessor),
    ("clf", LogisticRegression(max_iter=2000,class_weight="balanced",
                              random_state=RANDOM_STATE)),
])
cv_scores = cross_validate(pipe_lr, X, y,
    cv=cv,scoring=["precision","recall","f1","roc_auc"]
)
pd.DataFrame({k.replace("test_",""): [v.mean(),v.std()]
    for k,v in cv_scores.items() if k.startswith("test_")},index=["mean","std"])
```

Metric	Mean (5-fold CV)	Std Dev
Precision	0.799	0.023
Recall	0.837	0.047
F1 Score	0.817	0.025
ROC-AUC	0.752	0.027

7. Model Evaluation — Confusion Matrix & ROC

7.1 Best F1 Model: Decision Tree + SMOTE (for reference)

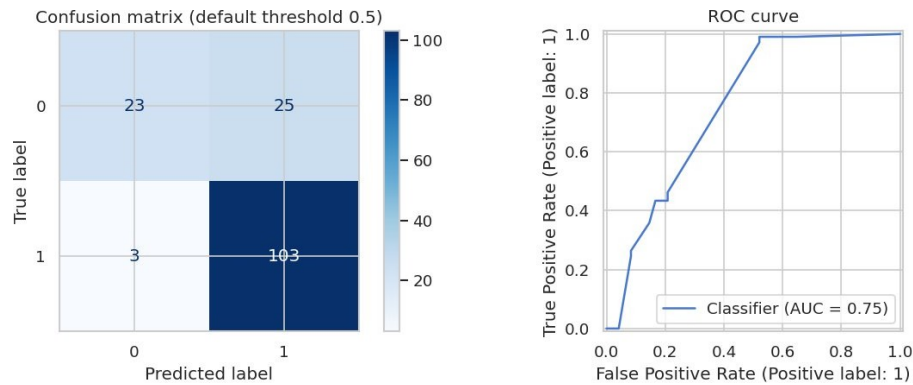


Fig 7a — Decision Tree + SMOTE: AUC=0.75; high recall but poor probability calibration

7.2 Recommended Deployment Model: Logistic Regression + class_weight

Cell 27 — Deploy model eval

```

deploy_strategy, deploy_name="class_weight", "LogisticRegression"
deploy_model, deploy_prep = fitted[(deploy_strategy,
deploy_name)]X_te = deploy_prep.transform(X_test)
y_pred = deploy_model.predict(X_te)
y_prob = deploy_model.predict_proba(X_te)[: ,1]

print(classification_report(y_test, y_pred,
target_names=["Rejected(0)", "Approved(1)"]))

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=axes[0], cmap="Blues")
axes[0].set_title("Confusion matrix (threshold = 0.5)")
RocCurveDisplay.from_predictions(y_test, y_prob, ax=axes[1])
axes[1].set_title("ROC curve - Logistic Regression")
plt.tight_layout()
plt.savefig(FIG_DIR / "07_deploy_model_eval.png", dpi=120)

```

precision	recall	f1-score	support	
Rejected (0)	0.73	0.73	0.73	48
Approved (1)	0.88	0.88	0.88	106
accuracy			0.84	154
macro avg	0.80	0.80	0.80	154
weighted avg	0.84	0.84	0.84	154

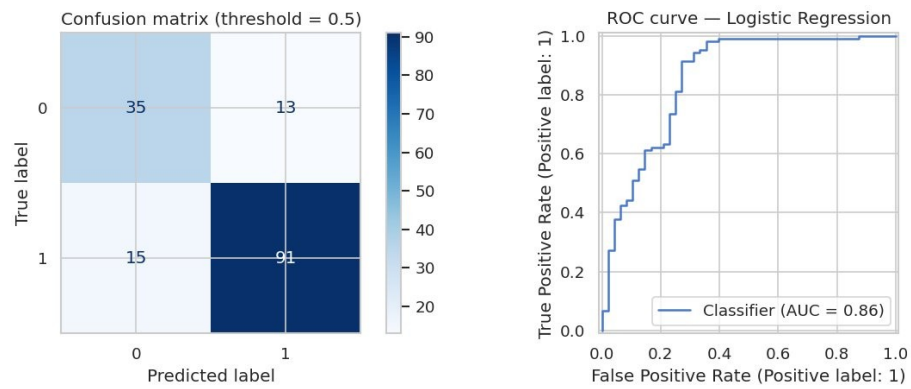


Fig 7b — Logistic Regression: AUC=0.86; well-calibrated probabilities, 35 TN / 91 TP

8. Threshold Optimisation

■ Cell 29 — Thresholdsweep

```
candidate_thresholds=np.arange(0.40,0.66,0.01) rows =
[]
for t incandidate_thresholds:
    pred = (y_prob >=t).astype(int)
    tn,fp,fn,tp=confusion_matrix(y_test,pred).ravel()
    reject_recall = tn / (tn + fn) if (tn + fn) else
    0rows.append({
        "threshold":      round(t,2),
        "precision":      precision_score(y_test,pred,zero_division=0),
        "recall_approved":recall_score(y_test, pred, zero_division=0),
        "recall_rejected":reject_recall,
        "f1":              f1_score(y_test,pred,zero_division=0),
        "approval_rate_pred": pred.mean(),
    })
thresh_df =pd.DataFrame(rows)

# Keep at least 50% recall on rejected class, then maximize F1
eligible = thresh_df[thresh_df["recall_rejected"] >= 0.50]
deploy_threshold=eligible.loc[eligible["f1"].idxmax(),"threshold"]
print(f"Selected threshold: {deploy_threshold}")
```

Selected threshold:0.40

■ Cell 30 — Threshold trade-offchart

```
fig, ax = plt.subplots(figsize=(8,4))
ax.plot(thresh_df["threshold"],thresh_df["precision"],          label="Precision(approved)")
ax.plot(thresh_df["threshold"],thresh_df["recall_approved"], label="Recall(approved)")
ax.plot(thresh_df["threshold"],thresh_df["recall_rejected"], label="Recall(rejected)")
ax.plot(thresh_df["threshold"], thresh_df["f1"],

label="F1")ax.axvline(deploy_threshold, color="red", linestyle="--",
        label=f"Deploy t={deploy_threshold:.2f}")
ax.set_xlabel("Probabilitythreshold(approveifP(approve)>=t)")ax.set_yl
abel("Score")
ax.set_title("Thresholdtrade-off-LogisticRegression")
ax.legend(loc="center left", bbox_to_anchor=(1,
0.5))plt.tight_layout()
```

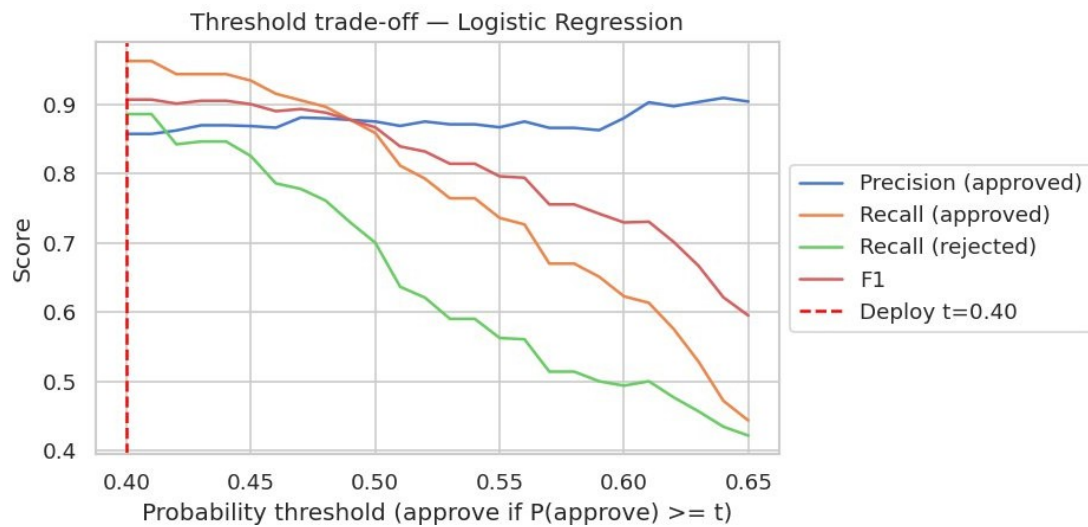


Fig 8 — At $t=0.40$ (red dashed line) F1 is maximised while recall on rejected class stays $\geq 50\%$

Score band	Decision	Rationale
$P(\text{approve}) \geq 0.55$	Auto-approve	High confidence — low false positive rate
$0.40 \leq P < 0.55$	Manual review	Borderline — route to loan officer
$P(\text{approve}) < 0.40$	Reject	Strong rejection signal from model

9. Repository & Deliverables

GitHub repo	https://github.com/abhinavgomra/Loan_approval_prediction
Notebook	https://github.com/abhinavgomra/Loan_approval_prediction/blob/main/internship/Zomato/loan/loan_approval_prediction.ipynb
Local path	internship/Zomato/loan/loan_approval_prediction.ipynb
Model report	MODEL_REPORT.md (trade-offs + threshold rationale)
Metrics CSV	outputs/model_metrics.csv
Threshold CSV	outputs/threshold_sweep.csv
This PDF	Loan_Approval_Submission.pdf